

Module 1: Lesson 3

Hyperparameter Optimization (HPO)

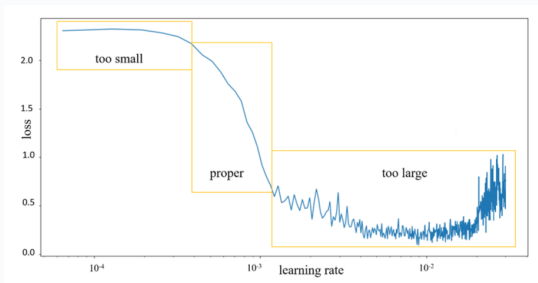


Outline

- ▶ Hyperparameter Optimization (HPO): why, what, and how?
- ▶ Which hyperparameters?
- ▶ Optimization algorithms

Hyperparameter Optimization (HPO)

- ▶ So far, we have used neural networks in a kind of **brute force** approach: simply by using trial and error in our models, we would see which combination of features (i.e., number of hidden layers, learning rate, optimizer, etc.) perform best.
- ▶ This is however quite inefficient because it requires spending a lot of time deciphering the best choices for a model.
- ▶ **Solution:** Hyperparameter Optimization (HPO)



Hyperparameter Optimization (HPO)

► **Solution:** Hyperparameter Optimization (HPO)

- Hyperparameters refer to parameters that cannot be updated during the training of the model.
- These hyperparameters can be **structure-related** (e.g., # of hidden layers), or **training-related** (e.g., learning rate or minibatch size)
- Final step of model design and first step of training a neural network

► **Purpose of HPO:**

1. Reduce costly menial work by the researcher
2. Improve accuracy and efficiency of training
3. Make the choice of hyper-parameters more convincing and reproducible

► **How?**

- We'll be using a series of optimization techniques
- These can be classified as **search algorithms** (e.g., grid search) and **trial algorithms** (e.g., curve fitting).

Which hyperparameters?

Given that HPO takes considerable computational resources, it is important to understand which hyperparameters take preferential treatment in the tuning. Here, we review major hyperparameters based on previous researchers' experience.

► Training-related hyperparameters:

- Learning rate (constant, linear/exponential decay)
- Optimizer (Mini-batch SGD, RMSprop, Adam)
- ...

► Model-design hyperparameters:

- Number of hidden layers
- Width (number of neurons) of hidden layers
- Regularization in cost function (ℓ_1 vs. ℓ_2 norms)
- Dropout (which rate?)
- Activation function (ReLU, Sigmoid, Tanh?)
- ...

Optimization algorithms

- Mathematically, HPO consists simply of finding a set of hyperparameters that achieve the minimum loss (or maximum accuracy) of a network/model.
- Computationally, HPO represents a complex computational problem to be solved by state-of-the-art algorithms of mainly two types: search and trial.

► Search algorithms

- Grid search: straightforward, but computationally costly (remember the curse of dimensionality!)
- Random search: similar to grid with randomized search (less time-consuming)
- Bayesian Optimization: selects next hyperparameter based on previous experience

► (Trial) Early-stopping algorithms

- Curve fitting: LPA algorithm (Learn, Predict, Assess)
- Successive Halving (SHA) and Hyperband (HB): random search sampling method + bandit-based early-stopping policy
- Extensions: Asynchronous Successive Halving (ASHA) and Bayesian Optimization Hyperband (BOHB)

Summary of Lesson 3

In Lesson 3, we have looked at:

- ▶ What is HPO, why we need it, and the basics of how to perform it
- ▶ Main hyper-parameters we can tune
- ▶ Basic review of algorithms for HPO

⇒ **References for this lesson:**

Yu, Tong, and Hong Zhu. "Hyperparameter Optimization: A Review of Algorithms and Applications." arXiv, 2020, <https://arxiv.org/abs/2003.05689>

. **TO-DO NEXT:** There is no associated Jupyter Notebook for this lesson, but it is extremely important that you go over the required readings for this lesson thoroughly.

In the next lesson, we will come back one more time to our stock timing example to see how a regression model based on a somewhat simple neural network can benefit from hyperparameter tuning using the Keras Tuner.